

CLOUD-05: Azure Integration

Series: CLOUD — Cloud Provider Integrations | **Notebook:** 5 of 8 | **Created:** March 2026 | **Last Updated:** 09/24/2026

Overview

This notebook covers Dynatrace's integration with Microsoft Azure. You will learn how to configure Azure Monitor metrics ingestion, which Azure services are supported, authentication via Azure AD (Entra ID), how resource group organization maps to Dynatrace monitoring, and how to query Azure entities and metrics with DQL.

Table of Contents

1. [Azure Integration Architecture](#)
 2. [Authentication Setup](#)
 3. [Supported Azure Services](#)
 4. [Querying Azure Entities](#)
 5. [Azure Metrics with DQL](#)
 6. [AKS Monitoring](#)
 7. [Resource Group Mapping](#)
 8. [Summary and Next Steps](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	<code>metrics.read</code> , <code>entities.read</code> , <code>ReadConfig</code> , <code>WriteConfig</code>
Azure Subscription	With Entra ID (Azure AD) app registration for Dynatrace
Connection	Azure Native Dynatrace Service (recommended) or Clouds app or Environment ActiveGate (classic)
Prior Knowledge	CLOUD-01 fundamentals

1. Azure Integration Architecture

Dynatrace offers multiple integration paths for Azure, with the **Azure Native Dynatrace Service** being the most streamlined for new deployments.

Connection Methods

Method	Mechanism	Best For
Azure Native Dynatrace Service	Fully managed via Azure Marketplace — no ActiveGate needed	New deployments, Azure-native teams
Clouds app (direct connection)	Connect via Entra ID app registration from Dynatrace SaaS	Existing Dynatrace SaaS environments
Classic ActiveGate polling	ActiveGate queries Azure Monitor API every 5 min	Managed deployments, legacy setups
Azure Event Hub	Streams metrics/logs via Event Hub	Near real-time log ingestion

Method	Mechanism	Best For
OneAgent code-level tracing	OneAgent on the VM, App Service, or container auto-instruments Azure SDK calls made by your application processes , producing distributed-trace spans rather than Azure Monitor resource metrics	Seeing <i>which code path</i> called <i>which Azure service</i> , and what it cost

Two different signal classes, not four options and a fifth. The first four rows are **resource-metric** paths — they pull what Azure Monitor already measures *about* an Azure resource. The last row is a **code-level tracing** path — it captures your application's own calls *into* Azure services. Most Azure estates need both; §3 draws the distinction in detail.

Azure Native Dynatrace Service

The Azure Native Dynatrace Service is available through the **Azure Marketplace** and provides:

- **Zero-infrastructure setup** — no ActiveGate or additional VMs required
- **Unified billing** through Azure Marketplace
- **One-click log forwarding** — subscription activity logs and resource logs forwarded directly
- **OneAgent auto-deployment** as an extension on VMs and App Services
- **SSO integration** with Entra ID

Data Flow (Modern)

```
Azure Resources → Azure Monitor → Dynatrace SaaS (direct connection)
                  → Diagnostic Settings → Dynatrace (logs via Event Hub or
                  direct)
```

Supported Azure Regions

Dynatrace supports all Azure public regions, Azure Government, and Azure China (with separate configuration).

Region values are programmatic strings, not display names. Azure regions surface in Dynatrace as lowercase, no-space identifiers — `eastus` , `westeurope` , `southcentralus` — not `East US` / `West Europe` . Filters, tag rules, and management-zone conditions must match the programmatic form (standardized as of OneAgent 1.337). A condition written against the display name silently matches nothing rather than erroring.

Release Radar (April 2026): the Clouds app extends to Microsoft Azure

The enhanced **Clouds app** experience — already available for AWS — now extends to **Microsoft Azure**, bringing Azure to parity. (The Release Radar entry says Dynatrace “*extends*” the experience; it does not use the lifecycle term **GA**, so neither does this entry.) For net-new Azure onboarding, this is the recommended path; the connection-mechanism table above still describes what runs underneath.

What the native Azure experience adds:

Capability	What it gives you
Unified resource view	Metrics, logs, metadata, and topology for Azure subscriptions in one place, alongside AWS
Opinionated insights & dashboards	Pre-built dashboards and investigations over enriched Azure telemetry to shorten root-cause analysis
Pre-configured health alerts	Health alerts created and managed directly in the Clouds app, with drill-down, search, and filtering scoped to Azure resources
Broad metric coverage	Any Azure Monitor native platform metric across services, queryable through DQL
Topology inventory	Periodic scanning enriched with native metadata (tags, subscription IDs), fully queryable via DQL
Onboarding & lifecycle management	Centralized provisioning that converts Azure subscriptions into native Dynatrace connections, reducing operational overhead

Note: Azure resource topology and metadata surface through the modern Smartscape model. The `dt.entity.azure_*` entity types in the queries below remain valid for entity lookups; for new topology-traversal queries prefer `dt.smartscape.*` and `smartscapeNodes` .

2. Authentication Setup

For non-native integrations, Azure uses an Entra ID (Azure AD) app registration. The Azure Native Dynatrace Service handles authentication automatically.

Setup Steps

1. **Create the Azure connection in Dynatrace** — it displays the issuer, subject and audience values the next step needs
2. **Register an application** in Entra ID (Azure Active Directory) and add a **federated identity credential** using those issuer, subject and audience values (or, where federation is not possible, a client secret)
3. **Assign the Monitoring Reader role** to the app's service principal at the subscription or management group level
4. **Update the connection in Dynatrace** with the Tenant ID and Client ID (plus the client secret, if you used one)

Required Azure Permissions

Scope	Role	Purpose
Subscription or Management Group	Monitoring Reader	The role the Azure connection docs assign to the service principal
Subscription or Management Group	Reader	Classic (Settings / ActiveGate) Azure integration

Authentication Methods Comparison

Method	Rotation	Recommended
Federated identity credential (OIDC token exchange; Clouds app connection)	None — no secret exists	Production (recommended)
Client secret	Manual; keep expiry < 12 months	Only where federation is not possible
Managed identity	Automatic	Classic ActiveGate running on an Azure VM only

Best Practice: Use the **Azure Native Dynatrace Service** for the simplest setup. For a Clouds app connection, use a **federated identity credential** — there is no secret to rotate. If using classic integration with an ActiveGate on an Azure VM, use **managed identity** to eliminate credential management.

Sources: [Create an Azure connection via CLI \(DT docs\)](#) — *"Federated identity credentials provide passwordless authentication and are more secure than client secrets."* The same page assigns the Monitoring Reader role and, for client secrets, notes that Microsoft recommends *"an expiration duration of less than 12 months for enhanced security"*.

3. Supported Azure Services

Dynatrace monitors 50+ Azure services. Key services:

Category	Services
Compute	Virtual Machines, VM Scale Sets, App Service, Azure Functions, Container Instances
Containers	AKS (Azure Kubernetes Service), Container Apps
Database	SQL Database, Cosmos DB, Database for PostgreSQL/MySQL, Cache for Redis
Storage	Blob Storage, File Storage, Queue Storage, Data Lake
Networking	Load Balancer, Application Gateway, Front Door, Traffic Manager, VPN Gateway
Integration	Service Bus, Event Hub, Event Grid, Logic Apps, API Management
AI/ML	Cognitive Services, Machine Learning, OpenAI Service

What this table is — and is not. These are the services covered by **Azure Monitor metrics** ingestion: Dynatrace polls Azure Monitor and stores the resource-level metrics Azure publishes about each service. That is coverage *of the resource*, not of your code's interaction with it. For **code-level visibility** — which application call reached Service Bus, how long a Cosmos DB read took, which span failed and why — you need **OneAgent** on the process making the call (see §1's *OneAgent code-*

level tracing row). The two are complementary: Azure Monitor tells you a resource is throttling; OneAgent tells you which service and code path is being throttled. Azure's own service list and Dynatrace's coverage both move; re-check the supported-services list against your tenant's Clouds app rather than treating the table above as exhaustive.

OneAgent Azure SDK Tracing (OneAgent 1.343)

Forthcoming / rolling out with [OneAgent 1.343](#), released 07/28/2026. Rollout is **per agent fleet, not per tenant** — **verify the OneAgent version on the hosts concerned**. The tenant version is not the agent version, and fleets commonly lag by more than one sprint.

Azure service	Automatic tracing added for	Note
Service Bus	Java (1.2.31+), Node.js, Python	Covers both sending and receiving messages
Event Hub	Java (1.2.26+), Node.js	Python is not part of this release
Cosmos DB	Python	Adds to the runtimes already supported

Azure Functions instrumentation widened in the same release:

Plan / OS	Runtime	Trigger types
Flex Consumption Plan (Linux)	Python	HTTP/webhooks, Service Bus, Event Hubs, Timer, Blob Storage, Event Grid, Azure SQL
Consumption / Premium / Dedicated (Windows)	Java, Node.js	HTTP/webhooks, Service Bus, Event Hubs, Timer

Re-baseline Cosmos DB request charges across this upgrade. OneAgent 1.343 corrected how `db.cosmosdb.request_charge` values are summarized on aggregated spans. Request-charge figures collected **before** 1.343 should be re-baselined rather than compared across the upgrade — a step change at the upgrade boundary is the correction landing, not a workload change.

On OneAgent 1.342 and earlier, the Azure Monitor resource metrics in the table above remain the only coverage for these services — the code-level spans are simply not

produced.

Entity Type Mapping

Azure Resource	Dynatrace Entity Type
Virtual Machine	dt.entity.azure_vm
Web App / App Service	dt.entity.azure_web_app
SQL Database	dt.entity.azure_sql_database
Cosmos DB	dt.entity.azure_cosmos_db
Load Balancer	dt.entity.azure_load_balancer
IoT Hub	dt.entity.azure_iot_hub

4. Querying Azure Entities

List Azure Virtual Machines

```
// List all monitored Azure VMs
fetch dt.entity.azure_vm, from:-7d
| fieldsKeep id, entity.name, tags
| sort entity.name asc
| limit 20

// Time range required (corrected 08/12/2026): dt.entity.* is an event-
LOOKBACK view – it returns
// only entities SEEN in the query window, not the standing inventory. Without
an explicit from:
// this under-counted against the notebook default window and still looked
like a valid answer.
// Smartscape note (dt.entity.* is deprecated but still functional): this
resource IS a Smartscape
// node – AWS_EC2_INSTANCE, AWS_RDS_DBINSTANCE,
AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES,
// AZURE_MICROSOFT_WEB_SITES (CloudFormation / ARM type, uppercased). On
Clouds-app connections the
// classic dt.entity.* type can under-count or return nothing for the same
estate (validation
// tenant 09/24/2026: RDS 0 classic vs 3 Smartscape; EC2 6 vs 25 running).
Prefer the Smartscape
// query.
```

Count Azure Resources by Type

```
// Count Azure VMs and Web Apps
fetch dt.entity.azure_vm, from:-7d
| summarize resource_count = count()
| fieldsAdd provider = "Azure", resource_type = "Virtual Machine"
| append [
    fetch dt.entity.azure_web_app, from:-7d
    | summarize resource_count = count()
    | fieldsAdd provider = "Azure", resource_type = "Web App"
]
| append [
    fetch dt.entity.azure_sql_database, from:-7d
    | summarize resource_count = count()
    | fieldsAdd provider = "Azure", resource_type = "SQL Database"
]
| sort resource_count desc

// Time range required (corrected 08/12/2026): dt.entity.* is an event-
LOOKBACK view – it returns
// only entities SEEN in the query window, not the standing inventory. Without
an explicit from:
// this under-counted against the notebook default window and still looked
like a valid answer.
// Smartscape note (dt.entity.* is deprecated but still functional): this
resource IS a Smartscape
// node – AWS_EC2_INSTANCE, AWS_RDS_DBINSTANCE,
AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES,
// AZURE_MICROSOFT_WEB_SITES (CloudFormation / ARM type, uppercased). On
Clouds-app connections the
// classic dt.entity.* type can under-count or return nothing for the same
estate (validation
// tenant 09/24/2026: RDS 0 classic vs 3 Smartscape; EC2 6 vs 25 running).
Prefer the Smartscape
// query.
```

List Azure Web Apps

```
// List monitored Azure App Service instances
fetch dt.entity.azure_web_app, from:-30d
| fieldsKeep id, entity.name, tags
| sort entity.name asc
| limit 20

// Smartscape note (dt.entity.* is deprecated but still functional): this
resource IS a Smartscape
// node – AWS_EC2_INSTANCE, AWS_RDS_DBINSTANCE,
AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES,
// AZURE_MICROSOFT_WEB_SITES (CloudFormation / ARM type, uppercased). On
Clouds-app connections the
// classic dt.entity.* type can under-count or return nothing for the same
estate (validation
// tenant 09/24/2026: RDS 0 classic vs 3 Smartscape; EC2 6 vs 25 running).
Prefer the Smartscape
// query.
```

5. Azure Metrics with DQL

Azure Monitor metrics are ingested into Dynatrace under the `cloud.azure.*` namespace.

Azure VM CPU Usage

```
// Azure VM CPU (Azure Monitor metric, via the Azure connection) – no OneAgent
needed.
//
// Corrected 09/24/2026: this cell used dt.host.cpu.usage with no provider
filter, which is the
// OneAgent host metric for EVERY host (EC2, Azure and Kubernetes nodes alike)
– not Azure VMs.
timeseries cpu =
avg(cloud.azure.microsoft_compute.virtualmachines.PercentageCPU), from:-6h,
by:{azure.resource.name}
| fieldsAdd avgCpuValue = arrayAvg(cpu)
| sort avgCpuValue desc
| limit 10
```

Azure VM Available Memory

```
// Azure VM available memory % (Azure Monitor metric), lowest first, over the
// last 6 hours.
//
// Corrected 09/24/2026: this cell used dt.host.memory.usage with no provider
// filter – the OneAgent
// metric for every host, not an Azure VM metric.
timeseries mem =
avg(cloud.azure.microsoft_compute.virtualmachines.AvailableMemoryPercentage),
from:-6h, by:{azure.resource.name}
| fieldsAdd avgAvailableMemPct = arrayAvg(mem)
| sort avgAvailableMemPct asc
| limit 10
```

6. AKS Monitoring

Azure Kubernetes Service (AKS) monitoring follows similar patterns to EKS (see CLOUD-03) with Azure-specific considerations.

AKS vs EKS Monitoring Differences

Aspect	AKS	EKS
Managed control plane	Yes (free tier)	Yes
Node agent	DynaKube Operator	DynaKube Operator
Native monitoring	Azure Monitor for containers	CloudWatch Container Insights
Network plugin	Azure CNI / Kubenet	VPC CNI
Serverless nodes	Virtual Nodes (ACI)	Fargate
Authentication	Entra ID (Azure AD) integration	IAM/IRSA

AKS Node Metrics

```
// AKS container CPU usage by namespace over the last hour
timeseries containerCpu = avg(dt.kubernetes.container.cpu_usage), from:-1h,
by:{k8s.namespace.name}
| fieldsAdd avgCpu = arrayAvg(containerCpu)
| sort avgCpu desc
| limit 10
```

AKS Pod Memory

```
// Container memory by namespace over the last hour
timeseries containerMem = avg(dt.kubernetes.container.memory_working_set),
from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgMemBytes = arrayAvg(containerMem)
| fieldsAdd avgMemMB = avgMemBytes / 1048576.0
| sort avgMemMB desc
| limit 10
```

7. Resource Group Mapping

Azure organizes resources into **resource groups**, which serve as logical containers. Dynatrace captures this organization for filtering and governance.

Mapping Strategy

Azure Construct	Dynatrace Equivalent	Purpose
Subscription	Tag / Management Zone	Top-level organizational boundary
Resource Group	Tag / Management Zone	Logical grouping for team/application
Tags	Dynatrace tags	Flexible metadata (environment, cost-center, team)

Best Practices

- **Map resource groups to Management Zones** (or Segments in Grail) for access control
- **Propagate Azure tags** to Dynatrace for consistent filtering
- **Use naming conventions** that include subscription, environment, and application:
`rg-myapp-prod-eastus`
- **Create alerting profiles** per resource group to route alerts to the right team

8. Summary and Next Steps

Key Takeaways

- Azure integration uses **Entra ID app registration** with the **Monitoring Reader** role; authenticate with a **federated identity credential** (recommended) rather than a client secret
- **Managed identity** is the most secure option when ActiveGate runs on an Azure VM
- **AKS monitoring** uses the same DynaKube Operator as EKS with Azure-specific networking considerations
- **Resource groups and Azure tags** should map to Dynatrace Management Zones/Segments for governance

Next Steps

- **CLOUD-06: GCP Integration** — Google Cloud monitoring setup
- **CLOUD-07: CloudWatch Log Ingestion** — Log forwarding patterns (applicable to Azure via Event Hub)
- **CLOUD-08: Multi-Cloud Patterns** — Unified monitoring across Azure and other providers

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.